Lab 04

TITLE

Docker, Python and Database Connections (MySQL and MongoDB)

Name:
Areeba Hameed

Registration number:
(2023-BSCS-15)

Section:
A

GitHub username:
Areebahameed936-dot

Contents

- Getting Started
- Task 1: Fork and Clone the Databases Repository
- Task 2: Install Docker, Docker Compose and Python
- Task 3: Start the Database Services
- Task 4: MySQL with Values in the Code
- Task 5: MongoDB with Values in the Code
- Task 6: MongoDB with os.getenv()
- Task 7: MongoDB with a Credentials File
- Summary

Getting Started

The evidence directory was created inside the existing **DevOps** repository. All programming work is done in the separate **Databases** fork, while screenshots and observations are kept in the DevOps repository.

```
cd ~/DevOps
mkdir -p Labs/Lab04/screenshots
```

The directory was created without errors (visible in the right-hand pane of Figure 1).

Task 1: Fork and Clone the Databases Repository

Steps

- Opened the **Databases** repository on GitHub and selected **Fork**, keeping the name *Databases*.
- Confirmed that the page shows *areebahameed936-dot/Databases* and, under the title, “forked from WaqasSaleem97/Databases”.
- Cloned my own fork on the Ubuntu server.

```
cd ~
git clone git@github.com:areebahameed936-dot/Databases.git
cd Databases
```

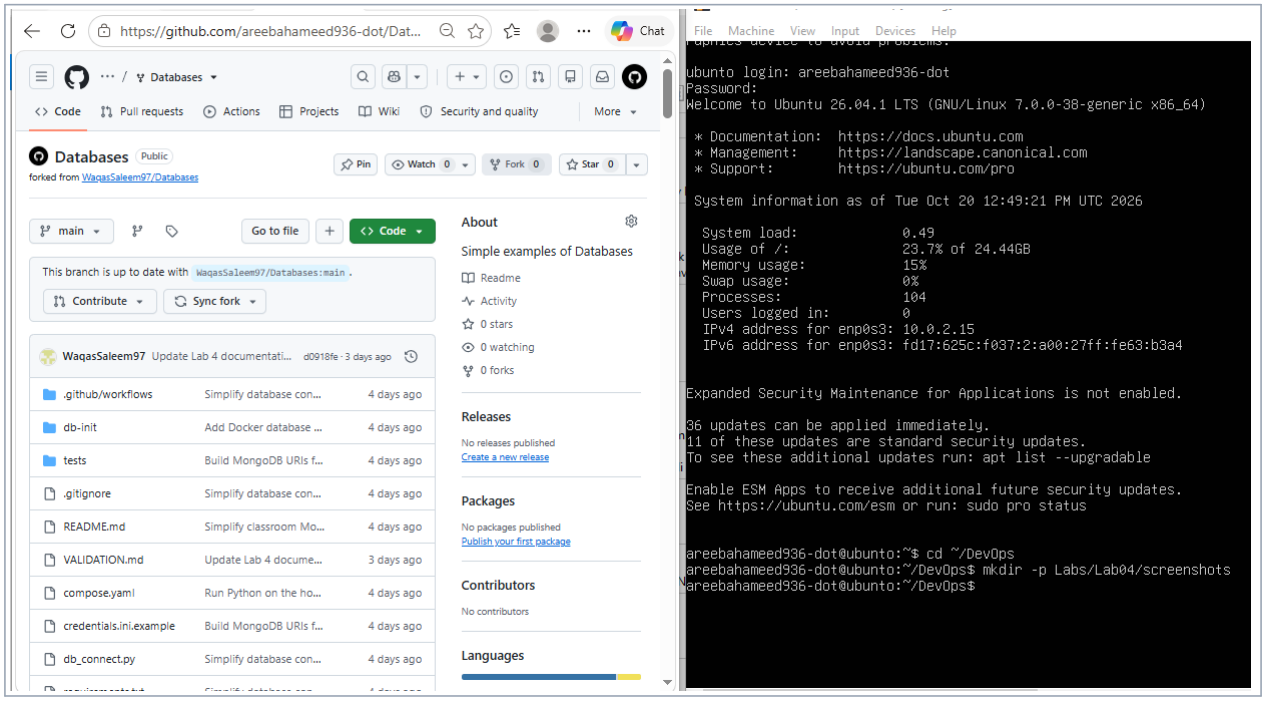


Figure 1: Split-screen: GitHub fork page (left) and Ubuntu terminal creating the evidence directory (right) `(databases_fork.png)`

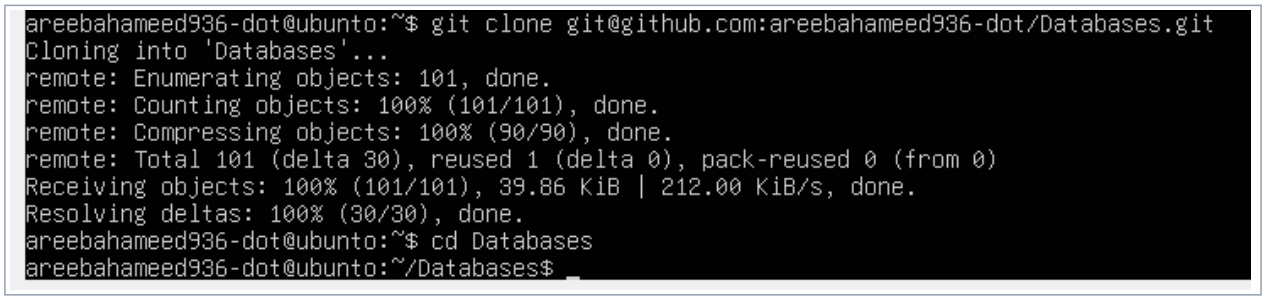


Figure 2: Cloning the fork into ~/Databases (101 objects received, deltas resolved) `(databases_clone.PNG)`

Observation

The fork lives under my own account and points back to the original repository as its source. The clone completed successfully over SSH, so the code that will be edited is my fork, while the separate DevOps repository holds the evidence.

Task 2: Install Docker, Docker Compose and Python on Ubuntu

Install Docker Engine, curl and the Compose plug-in

```
sudo apt update -y
sudo apt install -y docker.io curl

sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
     -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

sudo systemctl start docker
sudo usermod -aG docker $USER
exit
```

After signing out completely and reconnecting (so the new *docker* group membership takes effect), the three installations were verified:

```
ssh areebahameed936-dot@<ubuntu-server-ip>
git --version
docker --version
docker compose version
```



Figure 3: Git 2.53.0, Docker 29.1.3 and Docker Compose v5.6.1 installed (docker_installation.png)

Install Python and prepare the host environment

```
sudo apt install -y python3 python3-venv python3-pip
cd ~/Databases
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Observation

All three version commands print their versions, so Git, Docker and Docker Compose are ready. A Python virtual environment (`.venv`) isolates the lab dependencies; the prompt prefix `(.venv)` in later screenshots confirms it is active.

Task 3: Start the Database Services

The four supplied databases (one MySQL and three MongoDB instances) were started with Docker Compose:

```
docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
docker compose ps
```

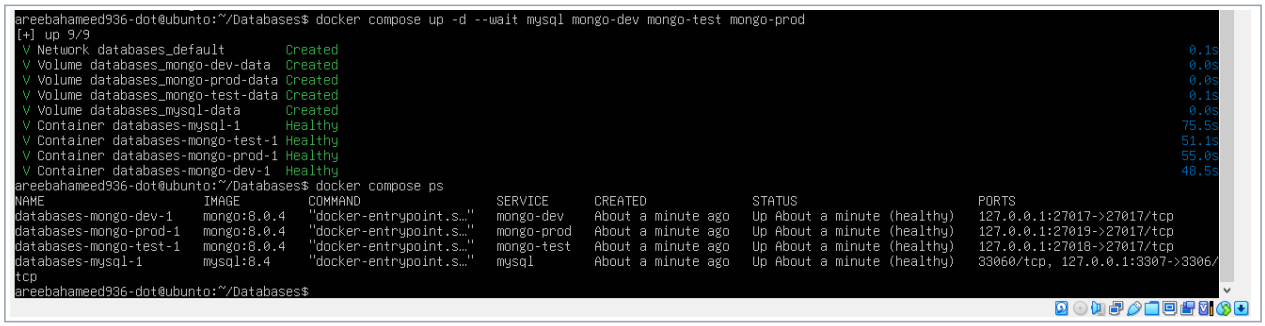


Figure 4: All four containers reported as Healthy by docker compose up --wait and docker compose ps (`database_services.PNG`)

The host ports used by Python were then inspected:

```
docker compose port mysql 3306
docker compose port mongo-dev 27017
docker compose port mongo-test 27017
docker compose port mongo-prod 27017
```



Figure 5: Host endpoints published for each database (`database_ports.png`)

Observation

Database	Container image	Host endpoint	Database name
MySQL	mysql:8.4	127.0.0.1:3307	school_mysql
MongoDB dev	mongo:8.0.4	127.0.0.1:27017	school_dev
MongoDB test	mongo:8.0.4	127.0.0.1:27018	school_test
MongoDB simulated prod	mongo:8.0.4	127.0.0.1:27019	school_prod

All four services are *healthy*. Each container publishes its internal port to a different port on the Ubuntu host (3306→3307, 27017→27017/27018/27019), which is why the Python code can reach them on `127.0.0.1`.

Task 4: MySQL with Values in the Code

`db_connect.py` contains a short MySQL connection whose host, port, user, password and database are written directly in the source:

```
import mysql.connector

connection = mysql.connector.connect(
    host="127.0.0.1",
    port=3307,
    user="mysql_student",
    password="MysqlDemoOnly123!",
    database="school_mysql",
)

print("Connected to MySQL:", connection.database)
print("Server:", connection.server_host, connection.server_port)
cursor = connection.cursor()
cursor.execute("SELECT environment FROM environment_info")
print("Environment:", cursor.fetchone()[0])
cursor.close()
connection.close()
```

```
python db_connect.py
```



Figure 6: MySQL connection with hard-coded values (`mysql_hardcoded_connection.png`)

Observation

- Output: **Connected to MySQL: school_mysql**, **Server: 127.0.0.1 3307**, **Environment: dev**.
- `user` and `password` supply the credentials, `host`/`port` identify the MySQL service and `database` selects the schema.
- The SQL query reads the environment name stored in the `environment_info` table; `cursor.close()` and `connection.close()` release the resources.
- **Summary:** the connection works, but its settings (including the password) are written directly in Python.

Task 5: MongoDB with Values in the Code

The contents of `db_connect.py` were replaced with a MongoDB version that still has the values in the code:

```
from pymongo import MongoClient

username = "dev_user"
password = "DevDemoOnly123!"
endpoint = "127.0.0.1:27017"
database_name = "school_dev"

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
print("Environment:", database.environment_info.find_one()["name"])
client.close()
```

```
python db_connect.py
```

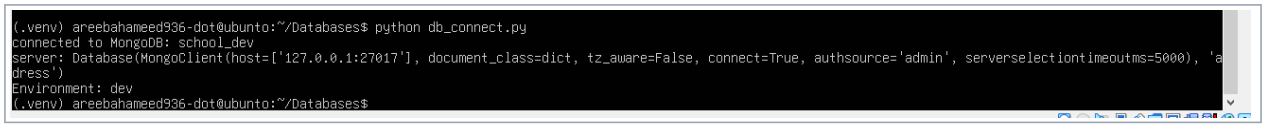


Figure 7: MongoDB (dev) connection with hard-coded values (`mongo_hardcoded_connection.png`)

Observation

- The connection reaches database **school_dev** on port **27017** and reports **Environment: dev**.
- `database.command("ping")` sends a real request to the server; creating a client or selecting a database alone does not prove connectivity.
- `serverSelectionTimeoutMS=5000` limits server selection to five seconds, so a wrong endpoint fails quickly.
- **Summary:** the database driver changed (MySQL → MongoDB) while the run command stayed the same.

Task 6: MongoDB with os.getenv()

The same file was replaced with a version that reads the four settings from environment variables, so the code itself no longer contains any credentials:

```
import os
from pymongo import MongoClient

username = os.getenv("MONGO_USERNAME")
password = os.getenv("MONGO_PASSWORD")
endpoint = os.getenv("MONGO_ENDPOINT")
database_name = os.getenv("MONGO_DATABASE")

if not username or not password or not endpoint or not database_name:
    raise SystemExit("Set all four MongoDB environment variables first.")

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
print("Environment:", database.environment_info.find_one()["name"])
client.close()
```

Dev environment

```
export MONGO_USERNAME=dev_user
export MONGO_PASSWORD='<dev password>'
export MONGO_ENDPOINT=127.0.0.1:27017
export MONGO_DATABASE=school_dev
clear
python db_connect.py
```

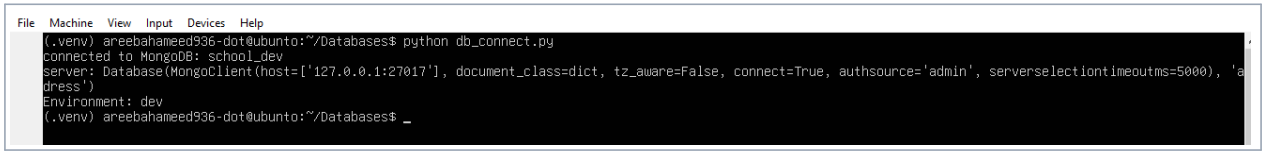


Figure 8: Environment-variable version connected to school_dev on port 27017 (environment: dev) (mongo_env_dev.png)

Test environment

```
export MONGO_USERNAME=test_user
export MONGO_PASSWORD='<test password>'
export MONGO_ENDPOINT=127.0.0.1:27018
export MONGO_DATABASE=school_test
clear
python db_connect.py
```

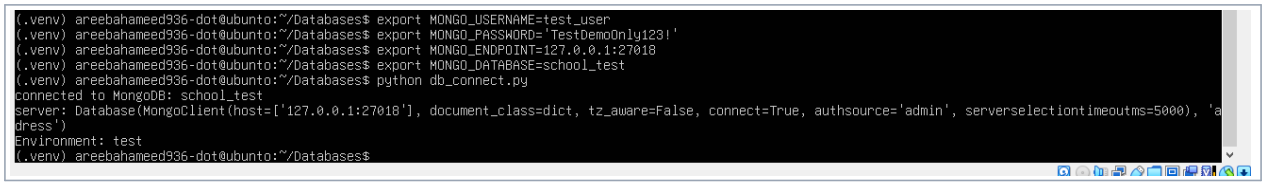


Figure 9: Same program connected to school_test on port 27018 (environment: test) (mongo_env_test.png)

Simulated production environment

```
export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='<prod password>'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
clear
python db_connect.py
```

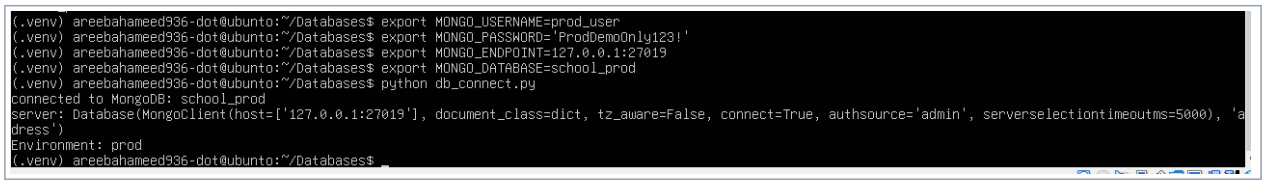


Figure 10: Same program connected to school_prod on port 27019 (environment: prod) (mongo_env_prod.png)

Observation

- Python inherits exported variables from the shell; the four variables supply separate values which Python joins into the connection URI.
- The `if` check stops the program with a clear message if any variable is missing.
- The code was **not changed** between runs – only the exported values were – yet it connected to three different databases (dev 27017, test 27018, prod 27019).
- **Summary:** one unchanged program connects to three databases using values from its process environment.

Task 7: MongoDB with a Credentials File

A local credentials file with three sections was created from the supplied example and then restricted to the owner:

```
cp -n credentials.ini.example credentials.ini
ls credentials.ini
```

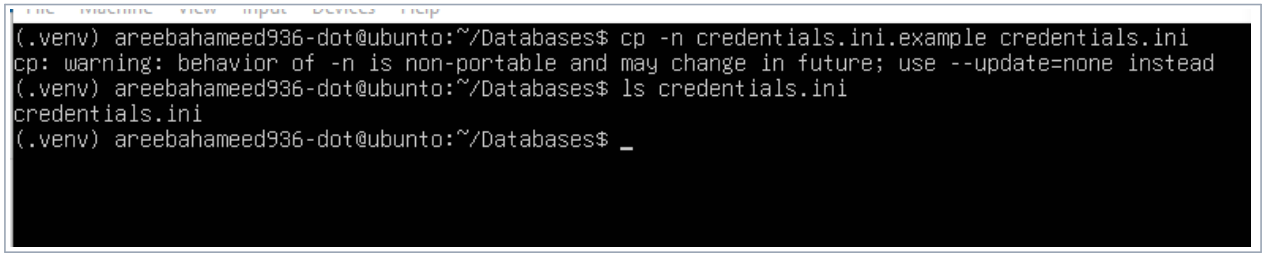


Figure 11: credentials.ini created from credentials.ini.example (credential_files_created.png)

```
chmod 600 credentials.ini
stat -c '%a %n' credentials.ini
```

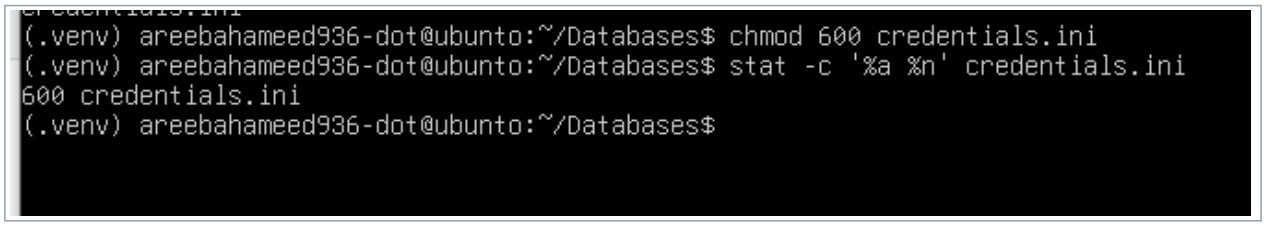


Figure 12: File mode 600 – readable and writable by the owner only (credential_file_permissions.png)

Structure of `credentials.ini` (one section per environment, each with username, password, endpoint and database):

```
[dev]
username = dev_user
password = <dev password>
endpoint = 127.0.0.1:27017
database = school_dev

[test]   # test_user, 127.0.0.1:27018, school_test
[prod]   # prod_user, 127.0.0.1:27019, school_prod
```

The final `db_connect.py` reads these settings with `configparser`:

```
import os
import configparser
from pymongo import MongoClient

config = configparser.ConfigParser(interpolation=None)
config.read("credentials.ini")
environment = os.getenv("APP_ENV", "dev")
username = config[environment]["username"]
password = config[environment]["password"]
endpoint = config[environment]["endpoint"]
database_name = config[environment]["database"]

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
print("Environment:", database.environment_info.find_one()["name"])
client.close()
```

Dev

```
export APP_ENV=dev
python db_connect.py
```



Figure 13: APP_ENV=dev → school_dev on 127.0.0.1:27017, environment dev (`mongo_file_dev.png`)

Test

```
export APP_ENV=test
python db_connect.py
```



Figure 14: APP_ENV=test → school_test on 127.0.0.1:27018, environment test (`mongo_file_test.png`)

Simulated prod

```
export APP_ENV=prod
python db_connect.py
```



Figure 15: APP_ENV=prod → school_prod on 127.0.0.1:27019, environment prod (`mongo_file_prod.png`)

Observation

- `os.getenv("APP_ENV", "dev")` selects the section name, defaulting to *dev*; `config[environment]` loads that section's four settings.
- An INI file stores values in plain text, so `chmod 600` (shown as **600 credentials.ini**) limits access to my account. `interpolation=None` makes the reader treat values such as "!" literally.
- The connection and output block are identical to the previous versions; only the source of the settings changed. This version is kept as the final `db_connect.py`.
- **Summary:** the same connection code now reads its settings from an external file.

Summary

Task	Method	Settings come from	Databases
4	mysql.connector	Hard-coded in the code	MySQL (dev)
5	pymongo	Hard-coded in the code	MongoDB dev
6	pymongo + os.getenv()	Shell environment variables	MongoDB dev / test / prod
7	pymongo + configparser	credentials.ini + APP_ENV	MongoDB dev / test / prod

Moving from hard-coded values to environment variables and then to an external, permission-restricted file keeps credentials out of the source code and lets one unchanged program target dev, test and prod. Environment variables are simple and per-process; a credentials file holds several environments in one place but stores them in plaintext, so its file permissions matter.